

Building Agentic AI Workflow with AWS Services

Use Case: "Accelerating Medical Research for Rare Diseases"

Scenario: A specialized medical research lab, "Synapse Innovations," is focused on finding new treatments for rare neurological disorders. Researchers often spend days or weeks sifting through new scientific publications, clinical trial updates, and internal research data to identify potential breakthroughs or refine existing hypotheses. They need a system that can rapidly synthesize information from diverse sources, prioritize relevant findings, and present actionable insights.

Functionality to be implemented:

- 1) Real time information on a certain disease or condition (user provided) will be obtained using Serper
- 2) A knowledge base containing research documents in PDF will be leveraged
- 3) Summary will be emailed to the researcher's email id
- 4) Automate the above workflow and execute it every 4 hours
- 5) Admin should be able to view and analyse the executions

Building the Agentic AI workflow with AWS Services

We will use the following AWS Services to create this flow:

- AWS Bedrock
- AWS Knowledgebases
- AWS Bedrock Claude Sonnet model
- Vector db
- Bedrock Agents
- Lambda functions

Implementation with AWS

Phase 1: Prerequisites & Initial Setup

Step 1: Get a Google Serper API Key

1. Go to **serper.dev**.
2. Sign up for a free account.
3. Navigate to your dashboard to find your API key.
4. Store this key securely. We will use AWS Secrets Manager for this.

Step 2: Store the Serper API Key in AWS Secrets Manager

1. In the AWS Console, navigate to **AWS Secrets Manager**.
2. Click **Store a new secret**.
3. For **Secret type**, choose **Other type of secret**.
4. Under **Key/value pairs**, set the key as SERPER_API_KEY and paste your key in the value field.
5. Click **Next**.
6. Give the secret a name, like prod/SerperApiKey, and a description.
7. Click **Next** and then **Store**. Note down the **Secret ARN**.

Step 3: Set up Amazon SES for Sending Emails

1. Navigate to **Amazon Simple Email Service (SES)** in the AWS console.
2. In the left menu, click **Verified identities**.
3. Click **Create identity**.
4. Select **Email address**.
5. Enter the email address you want to send reports *from* (e.g., research-bot@your-domain.com or your personal Gmail for this demo).
6. Click **Create identity**. You will receive a verification email. Click the link in it to verify ownership.
7. **Important:** For a demo, your SES account will be in a "sandbox" environment. This means you can *only* send emails to other verified identities. So, repeat steps 3-6 to also verify the recipient's email address (the researcher's Gmail ID).

Phase 2: Building the Knowledge Core (The Knowledge Base)

Step 1: Create the Knowledge Base

1. Navigate to **Amazon Bedrock** in the AWS Console.
2. Select **Knowledge Bases** from the left menu.
3. Click **Create Knowledge Base**.

Step 2: Provide Knowledge Base details

- **Name:** Synapse_Innovations_Research_DB

- An IAM role will be created automatically. Click **Next**.

Step 3: Set up Data source

- Click **Create and use a new data source**.
- **S3 URI:** Create a new S3 bucket (e.g., synapse-innovations-docs), upload your PDF research articles there, and then provide the S3 URI for that bucket.
- Click **Next**.

Step 4: Set up vector database

- Select **Quick create a new vector store**. Bedrock will automatically create an **Amazon OpenSearch Serverless** vector collection.
- Click **Next**.

Step 5: Review and Sync

- Review the configuration and click **Create Knowledge Base**.
- Once created, go to the Knowledge Base details page and click the **Sync** button to start indexing your documents.

Phase 3: Creating the Agent's Tools (Lambda Functions)

Tool 1: Lambda Function for Real-time Web Search

1. Navigate to **AWS Lambda** and click **Create function**.
2. Select **Author from scratch**.
 - **Function name:** BedrockAgent_SerperSearch
 - **Runtime:** Python 3.12
 - **Permissions:** Select **Create a new role with basic Lambda permissions**.
3. Click **Create function**.
4. **Add Permissions:** Go to the **Configuration > Permissions** tab, click the **Role name**, and in the IAM screen, attach the **SecretsManagerReadWrite** policy.
5. **Add Code:** In the **Code source** tab, paste the following Python code:

```
import json
import boto3
import requests

def get_secret():
```

```

session = boto3.session.Session()
client = session.client(service_name='secretsmanager')
secret_name = "prod/SerperApiKey"
get_secret_value_response =
client.get_secret_value(SecretId=secret_name)
return
json.loads(get_secret_value_response['SecretString'])['SERPER_API_KEY']

def lambda_handler(event, context):
    agent_event = event.get('requestBody', {})
    content = agent_event.get('content', {})
    properties = content.get('application/json', {}).get('properties',
[])
    search_query = ""
    for prop in properties:
        if prop.get('name') == 'search_string':
            search_query = prop.get('value')
            break
    if not search_query:
        return { 'response': 'Search query not provided.' }

    API_KEY = get_secret()
    url = "https://google.serper.dev/search"
    payload = json.dumps({"q": search_query})
    headers = {'X-API-KEY': API_KEY, 'Content-Type': 'application/json'}

    response = requests.request("POST", url, headers=headers,
data=payload)
    search_results = response.json().get('organic', [])
    formatted_results = [{'title': item.get('title'), 'link':
item.get('link'), 'snippet': item.get('snippet')} for item in
search_results[:5]]
    response_body = {"results": formatted_results}

    action_response = {
        'actionGroup': event.get('actionGroup', ''),
        'apiPath': event.get('apiPath', ''),
        'httpMethod': event.get('httpMethod', ''),
        'statusCode': 200,
        'responseBody': {'application/json': {'body':
json.dumps(response_body)}}
    }
    return action_response

```

6. **Add Requests Layer:** Go to **Layers > Add a layer**. Select **AWS Layers** and choose an AWS-provided layer containing the requests library (e.g., AWSLambda-Python312-SciPy1-Requests).

6. Click **Deploy**.

Tool 2: Lambda Function for Sending Email Reports

1. Create another Lambda function named BedrockAgent_EmailSender.
2. In its IAM role, attach the **AmazonSESFullAccess** policy.

3. **Add Code:** Paste the following code and **change the SENDER_EMAIL variable** to your verified SES email.

```
import json
import boto3

SES_CLIENT = boto3.client('ses')
SENDER_EMAIL = "your-verified-sender-email@example.com" # CHANGE THIS

def lambda_handler(event, context):
    agent_event = event.get('requestBody', {})
    content = agent_event.get('content', {})
    properties = content.get('application/json', {}).get('properties',
    [])

    recipient_email, subject, body = "", "", ""
    for prop in properties:
        if prop.get('name') == 'recipient': recipient_email =
prop.get('value')
        elif prop.get('name') == 'subject': subject = prop.get('value')
        elif prop.get('name') == 'body': body = prop.get('value')

    if not all([recipient_email, subject, body]):
        return {'response': 'Missing parameters.'}

    SES_CLIENT.send_email(
        Source=SENDER_EMAIL,
        Destination={'ToAddresses': [recipient_email]},
        Message={'Subject': {'Data': subject}, 'Body': {'Html': {'Data':
body.replace("\n", "<br>")}}})

    response_body = {"status": "Email sent successfully."}

    action_response = {
        'actionGroup': event.get('actionGroup', ''),
        'apiPath': event.get('apiPath', ''),
        'httpMethod': event.get('httpMethod', ''),
        'httpStatusCode': 200,
        'responseBody': {'application/json': {'body':
json.dumps(response_body)}}
    }
    return action_response
```

4. Click **Deploy**.

Phase 4: Building the AI Agent

Step 1: Create the Agent

1. Navigate back to **Amazon Bedrock > Agents** and click **Create Agent**.
 - **Name:** Medical_Research_Assistant
 - Select **Create and use a new service role**.

- **Select model:** Choose **Anthropic Claude 3 Sonnet**.
- **Instructions for the Agent:** This is the most critical part. Enter a detailed prompt that tells the agent how to behave.
- You are a world-class biomedical research assistant named Synapse. Your goal is to accelerate research for rare neurological disorders.
- Your process is as follows:
 - 1. When given a disease name, you **MUST** first use the Web Search tool to get the latest real-time news, publications, and clinical trial updates.
 - 2. Second, you **MUST** search the internal Knowledge Base for any relevant proprietary research findings on the same topic.
 - 3. Third, you **MUST** synthesize ALL the information from both the web search and the knowledge base into a concise, well-structured summary report. The report should have two sections: "External Findings (Web)" and "Internal Research (Knowledge Base)".
 - 4. Finally, you **MUST** use the Email Sender tool to email this complete summary to the specified recipient. Do not take shortcuts. Complete all steps.
- Click **Next**.

Step 2: Add Action Groups (Tools)

1. Create an Action group for the BedrockAgent_SerperSearch Lambda and paste its OpenAPI schema.
2. Create a second Action group for the BedrockAgent_EmailSender Lambda and paste its OpenAPI schema.
3. Click **Next**.

Step 3: Add Knowledge Base

1. Select the Synapse_Innovations_Research_DB you created.
2. Add the instruction: Search this knowledge base for internal research, lab notes, and proprietary findings.
3. Click **Next**.

Step 4: Review and Create

1. Review all details and click **Create Agent**.
-

Phase 5: Manual Testing

1. In the Bedrock console, with your new agent selected, use the testing window.
 2. Type a prompt: Please perform research on Spinocerebellar Ataxia and email the report to my-verified-email@gmail.com.
 3. Press **Run**.
 4. Click **Show trace** to see the agent's reasoning process and debug if necessary.
 5. Check your Gmail for the report.
-

Phase 6: Automating the Workflow

1. Navigate to **Amazon EventBridge > Schedules** and click **Create schedule**.
2. **Schedule name:** Synapse_Research_4_Hour_Run
3. **Schedule pattern:** Select **Recurring schedule** and configure it to run every **4 Hours**.
4. **Select target:** Search for and select **Bedrock**.
5. **InvokeAgent:** Choose your Medical_Research_Assistant agent.
6. **Input:** In the JSON input box, provide the static prompt for the scheduled run.

JSON

```
{  
  
  "inputText": "Please perform research on Huntington's Disease and email the report to  
your-verified-recipient@gmail.com."  
  
}
```

7. **Permissions:** Select **Create new role for this schedule**.
 8. Review and **Create schedule**.
-

Phase 7: Monitoring and Analysis

1. **Agent Traces:** Use the test window in the **Amazon Bedrock > Agents** console to see the detailed reasoning trace for each step of an execution.
2. **Lambda Logs:** Navigate to **Amazon CloudWatch > Log groups**. Find the log groups for your Lambda functions (e.g., /aws/lambda/BedrockAgent_SerperSearch) to see outputs and errors.

3. **EventBridge Monitoring:** In the EventBridge Scheduler, view the history of invocations to see if they succeeded or failed.